COMP5712M: Programming for Data Science

Coursework 1

@: dwqz0119/ Sakshi Kishore

Overview

This coursework tests your ability to define algorithmic functions in Python, focusing on data processing and transformation. Completing these tasks provides a foundational grounding in the essential programming techniques required for data science. Please complete Question 1 and Question 2 without engaging with any Generative AI (GenAI) tools. For Question 3, you will have the opportunity to do so and you have to write a comparative reflection by evaluating you solutions against those produced by a GenAI tool of your choice.

Please follow the following rules:

1. **No Additional Imports**: Do not add any new `import` statements. Use only the packages already provided in the notebook. Relying on external pre-programmed functions defeats the purpose of learning these algorithms.
2. **Preserve Templates**: You are given bare skeleton functions with the function name, arguments and a dummy return value. Edit the skeleton functions so they return the correct, specified outputs for any valid input types.
3. **Local testing**: For questions 1 and 2, you will be able to test your functions locally, by running testing functions defined in the file `cwk1_tests.py`. You should download this from Minerva and put it in the same directory as `cwk1.ipynb`. The local test cases are sample checks. Final grading will use a different, comprehensive set of unseen test cases. Ensure your functions satisfy the general requirements rather than just passsing the local samples.

Importing the Testing Module

Run the cell below to import the testing module and check your code against the sample function tests for Question 1 and Question 2.

In [47]:
```
from cwk1_tests import do_tests, tests_version

tests_version()
```

Out[47]:
```
(('Autograder:', 3.0), ('Tests:', '14 October 2026'))
```

Question 1: A Simple Movie Recommendation System (6 Marks)

You are asked to write functions for a simple system for recommending movies based on a user's maximum subscription tier budget and their desired genre/feature tags. These requirements will be compared to data about available movies in order to find those that match the cost and tag requirements. **You are not allowed to use GenAI to generate direct solutions and code for this question. All submitted solutions must be 100% your own independent work.**

Q1 (a) Find available movie tags (3 marks)

Write a function `available_movie_tags` that takes two input parameters:

- The maximum rental cost (represented as an integer) that a user is prepared to pay.
- A movie data dictionary.
  This dictionary maps the name of a movie (the key) to a list containing the rental cost (integer) and a list of genre/feature tags (strings).

In [48]:
```
MOVIES_EG = {
    "Toy Story": [2, ["animation", "family"]],
    "The Matrix": [3, ["sci-fi", "action"]],
    "Inception": [4, ["sci-fi", "action", "mind-bending"]],
    "Spirited Away": [4, ["animation", "fantasy"]],
    "The Godfather": [5, ["drama", "crime"]],
    "Pulp Fiction": [4, ["drama", "crime"]],
    "Interstellar": [5, ["sci-fi", "drama", "space"]],
    "Parasite": [6, ["drama", "thriller"]]
}
```

NOTE: `MOVIES_EG` is just an example of a movie data dictionary. Your function should work with any similar structure containing different movies, costs, and tags.

The value returned by the function `available_movie_tags` should be a list of all unique movie tags available from any movie whose cost is less than or equal to the maximum cost specified. This list must be sorted in *alphabetical order*.

Example Expected Outputs using `MOVIES_EG` dictionary.

Parameter 1 (cost)	Parameter 2 (movie data)	Return (tag list)
2	MOVIES_EG	['animation', 'family']
3	MOVIES_EG	['action', 'animation', 'family', 'sci-fi']
10	MOVIES_EG	['action', 'animation', 'crime', 'drama', 'family', 'fantasy', 'mind-bending', 'sci-fi', 'space', 'thriller']

In [49]:
```
def available_movie_tags(max_rent: int, movie_data: dict) -> list:
    tags = set()

    for movie in movie_data:
        rent_cost = movie_data[movie][0]
        movie_tags = movie_data[movie][1]

        if rent_cost <= max_rent:
```

```
            for tag in movie_tags:
                tags.add(tag)

    return sorted(tags)
```

In [50]:
```
# Run this cell to test your available_movie_tags function
do_tests(available_movie_tags)
```

```
*Autograder (v3.0)*
Testing function: available_movie_tags
Evaluating: available_movie_tags(2, MOVIES_TEST) ...
Returned: ['animation', 'family']
Expected answer: ['animation', 'family']
CORRECT :)    1 mark
Evaluating: available_movie_tags(3, MOVIES_TEST) ...
Returned: ['action', 'animation', 'family', 'sci-fi']
Expected answer: ['action', 'animation', 'family', 'sci-fi']
CORRECT :)    1 mark
Evaluating: available_movie_tags(6, MOVIES_TEST) ...
Returned: ['action', 'animation', 'crime', 'drama', 'family', 'fantasy', 'mind-be
nding', 'sci-fi', 'space', 'thriller']
Expected answer: ['action', 'animation', 'crime', 'drama', 'family', 'fantasy',
'mind-bending', 'sci-fi', 'space', 'thriller']
CORRECT :)    1 mark
--------------------------------------------------
Total mark for 'available_movie_tags' is 3 out of 3
--------------------------------------------------
```

Q1 (b) Find possible movie recommendations (3 marks)

Write a function `recommend_movies` that takes three input parameters:

- The maximum rental cost (represented as an integer).
- A list of desired tag strings.
- A movie data dictionary (same structure as Q1 (a)).

The value returned by `recommend_movies` should be a nested list of movies that satisfy all requirements (cost is less than or equal to max cost, and the movie contains all of the desired tags).

Each matching movie in the returned list should be represented as a sublist containing [movie_name, cost, [tags]]. This final list must be sorted in descending order of cost.

If the list of desired tags contains any string that is not a tag of any movie in the movie data dictionary, the value returned will be the empty list, `[ ]`, since no movies will match that requirement.

In [51]:
```
MOVIES_TEST = {
    "Toy Story": [2, ["animation", "family"]],
    "The Matrix": [3, ["sci-fi", "action"]],
    "Inception": [4, ["sci-fi", "action", "mind-bending"]],
    "Spirited Away": [4, ["animation", "fantasy"]],
    "The Godfather": [5, ["drama", "crime"]],
    "Pulp Fiction": [4, ["drama", "crime"]],
    "Interstellar": [5, ["sci-fi", "drama", "space"]],
    "Parasite": [6, ["drama", "thriller"]],
```

```
    "The Dark Knight": [7, ["action", "crime", "drama"]],
    "Spider-Man: Into the Spider-Verse": [8, ["animation", "action", "sci-fi"]],
    "Get Out": [9, ["horror", "mystery", "thriller"]],
    "Whiplash": [10, ["drama", "music"]],
    "Grand Budapest Hotel": [11, ["comedy", "adventure"]],
    "Alien": [12, ["sci-fi", "horror"]],
    "Paddington 2": [13, ["animation", "family", "comedy"]],
    "Everything Everywhere All at Once": [14, ["sci-fi", "action", "comedy", "mi
}
```

Example Expected Outputs using `MOVIES_TEST` dictionary.

max_cost	tags	movie_data	Return
5	["sci-fi", "action"]	MOVIES_TEST	[["Inception", 4, ["sci-fi", "action", "mind-bending"]], ["The Matrix", 3, ["sci-fi", "action"]]]
3	["sci-fi"]	MOVIES_TEST	[["The Matrix", 3, ["sci-fi", "action"]]]
5	["horror"]	MOVIES_TEST	[]

In [52]:
```
def recommend_movies(max_cost: int, tags: list, movie_data: dict) -> list:
    result = []


    available_tags = []
    for movie in movie_data:
        for tag in movie_data[movie][1]:
            if tag not in available_tags:
                available_tags.append(tag)

    for tag in tags:
        if tag not in available_tags:
            return []


    for movie in movie_data:
        cost = movie_data[movie][0]
        movie_tags = movie_data[movie][1]

        if cost <= max_cost and all(tag in movie_tags for tag in tags):
            result.append([movie, cost, movie_tags])


    for i in range(len(result)):
        for j in range(i + 1, len(result)):
            if result[i][1] < result[j][1]:
                result[i], result[j] = result[j], result[i]

    return result
```

In [53]:
```
# Run this cell to test your recommend_movies function
do_tests(recommend_movies)
```

```
*Autograder (v3.0)*
Testing function: recommend_movies
Evaluating: recommend_movies(5, ['sci-fi', 'action'], MOVIES_TEST) ...
Returned: [['Inception', 4, ['sci-fi', 'action', 'mind-bending']], ['The Matrix',
3, ['sci-fi', 'action']]]
Expected answer: [['Inception', 4, ['sci-fi', 'action', 'mind-bending']], ['The M
atrix', 3, ['sci-fi', 'action']]]
CORRECT :)    1 mark
Evaluating: recommend_movies(5, ['sci-fi', 'mind-bending'], MOVIES_TEST) ...
Returned: [['Inception', 4, ['sci-fi', 'action', 'mind-bending']]]
Expected answer: [['Inception', 4, ['sci-fi', 'action', 'mind-bending']]]
CORRECT :)    1 mark
Evaluating: recommend_movies(7, ['crime', 'drama'], MOVIES_TEST) ...
Returned: [['The Dark Knight', 7, ['action', 'crime', 'drama']], ['The Godfathe
r', 5, ['drama', 'crime']], ['Pulp Fiction', 4, ['drama', 'crime']]]
Expected answer: [['The Dark Knight', 7, ['action', 'crime', 'drama']], ['The God
father', 5, ['drama', 'crime']], ['Pulp Fiction', 4, ['drama', 'crime']]]
CORRECT :)    1 mark
------------------------------------------------
Total mark for 'recommend_movies' is 3 out of 3
------------------------------------------------
```

Question 2: Using a Data File of English Words (11 marks)

For this task you will deal with a file contains contains nearly all common English words:

- english_words.txt

The following code initialises the global variable `ENGLISH_WORDS` to be a **set** of all the words in the file `english_words.txt`, to be used for the rest of the tasks. You will need to have that file in the same folder as this notebook file.

In [54]:
```
def get_english_words():
    with open("english_words.txt") as f:
         words = f.readlines()
    words = { word.strip() for word in words }
    return words

ENGLISH_WORDS = get_english_words()
```

You are recommended to use the `ENGLISH_WORDS` variable in the following questions, whenever you are required to perform a calculation involving all the words in `english_words.txt`. This is particularly important for a function that needs to check the list of words many times. Reading information from a file typically takes more computational time than other operations, so if data can be stored in memory (e.g. as the value of a Python variable) this will usually be a lot more efficient than reading it from a file each time it is needed. (Of course, when dealing with very large datasets, it may not be possible to store the whole dataset in memory.) **You are not allowed to use GenAI to generate direct solutions and code for this question. All submitted solutions must be 100% your own independent work.**

Q2 (a) Check whether a string is an English word (2 marks)

Write a function `is_english_word(s)`, which will test if its input string `s` is an English word, according to the file english_words.txt. This file contains a large number of English words, including all common words and many very rare words. Proper names are not included, and all words are given in lower case, with one word on each line of the file. You need to download the file of English words. Note that it is not a program file and you do not need to edit it.

Your function `is_english_word(s)` should take any string as its argument and return a Boolean value --- i.e. `True` or `False`. More specifically your function should return `True` if *any* of the following conditions hold:

- The input string is one of the words in `english_words.txt`.
- The input string is the same as one of the of the words in `english_words.txt` except that the input string starts with a capital letter (with all the other letters being small).
- The input string is the same as one of the words in `english_words.txt` except that the input string is all in capital letters.

If none of these conditions hold, your function should return `False`.

Examples:

INPUT	OUPUT
`"python"`	`True`
`"Python"`	`True`
`"PYTHON"`	`True`
`"pyThon"`	`False`
`"splap"`	`False`

In [55]:
```
def is_english_word(s:str) -> bool:
    # complete your code here
    if s.lower() in ENGLISH_WORDS:
        if s == s.lower() or s == s.capitalize() or s == s.upper():
            return True
    return False
```

In [56]:
```
# Run this cell to test your is_english_word function
do_tests(is_english_word)
```

```
*Autograder (v3.0)*
Testing function: is_english_word
Evaluating: is_english_word("I") ...
Returned: True
Expected answer: True
CORRECT :)    1 mark
Evaluating: is_english_word("Python") ...
Returned: True
Expected answer: True
CORRECT :)    1 mark
Evaluating: is_english_word("HelP") ...
Returned: False
Expected answer: False
CORRECT :)    1 mark
Evaluating: is_english_word("Abecedarians") ...
Returned: True
Expected answer: True
CORRECT :)    1 mark
Evaluating: is_english_word("Brexit") ...
Returned: False
Expected answer: False
CORRECT :)    1 mark
-----------------------------------------------
Total mark for 'is_english_word' is 5 out of 5
-----------------------------------------------
```

Q2 (b) Literate Password Checker (4 marks)

Computer systems are vulnerable to hacking if they can be accessed by passwords based on English words. Now that we have access to the set of English words, we can define a tougher password checker.

An institution uses the following rules to classify the strength of passwords:

- A string is an ILLEGAL password if either:
  - it is an English word (as defined above)
  - it contains any invisible character (space, tab, newline)
- A string is a WEAK password if it is **not** ILLEGAL and, either:
  - it is *less than* 8 characters long.
  - it is an English word followed by one or more decimal digit characters
- A string is a STRONG password if it is **not** ILLEGAL and:
  - it contains at least 12 characters
  - AND it contains at least 1 lower case letter
  - AND it contains at least 1 capital letter
  - AND it contains at least 1 numerical digit
  - AND it contains at least 1 special character (any visible ASCII character that is not a letter or a number)
- A string is a MEDIUM password if it is **not** an ILLEGAL password and is **not** a WEAK password and is **not** a STRONG password.

You need to code a function `password_strength(s)` that will take a string argument and will return the 'strength' of that string as a password, according to the rules given above. So it should return one of the strings `"ILLEGAL"`, `"STRONG"`, `"WEAK"` or `"MEDIUM"`.

You may assume that the input password is consists of only ASCII characters, that is: alphabetic letters, numerical digits, special visible charactes, spaces, tabs and newlines. The special visible characters are:

```
~`!@#$%^&*(){}[]|\/:;"<>.?
```

Examples:

INPUT	OUPUT
`"secret"`	`"ILLEGAL"`
`"my secret"`	`"ILLEGAL"`
`"qwertyu"`	`"WEAK"`
`"hello123"`	`"WEAK"`
`"7Kings8all9Pies!"`	`"STRONG"`
`"brandon123"`	`"MEDIUM"`

In [57]:

```
def password_strength(s: str) -> str:

    if is_english_word(s):
        return "ILLEGAL"

    if " " in s or "\t" in s or "\n" in s:
        return "ILLEGAL"


    if len(s) < 8:
        return "WEAK"

    word = s.rstrip("0123456789")
    digits = s[len(word):]

    if digits != "" and is_english_word(word):
        return "WEAK"


    has_lower = False
    has_upper = False
    has_digit = False
    has_special = False

    for char in s:
        if char.islower():
            has_lower = True
        elif char.isupper():
            has_upper = True
```

```
        elif char.isdigit():
            has_digit = True
        else:
            has_special = True


    if (len(s) >= 12 and has_lower and has_upper
            and has_digit and has_special):
        return "STRONG"


    return "MEDIUM"
```

In [58]:
```
# Run this cell to test the password_strength function
do_tests(password_strength)
```

```
*Autograder (v3.0)*
Testing function: password_strength
Evaluating: password_strength("bananaIsABerry!") ...
Returned: 'MEDIUM'
Expected answer: 'MEDIUM'
CORRECT :)    1 mark
Evaluating: password_strength("Santo") ...
Returned: 'ILLEGAL'
Expected answer: 'ILLEGAL'
CORRECT :)    1 mark
Evaluating: password_strength("secret99") ...
Returned: 'WEAK'
Expected answer: 'WEAK'
CORRECT :)    1 mark
Evaluating: password_strength("Secret999!") ...
Returned: 'MEDIUM'
Expected answer: 'MEDIUM'
CORRECT :)    1 mark
Evaluating: password_strength("7Kings8all9Pies!") ...
Returned: 'STRONG'
Expected answer: 'STRONG'
CORRECT :)    1 mark
------------------------------------------------
Total mark for 'password_strength' is 5 out of 5
------------------------------------------------
```

Q2 (c) Word Length Distribution and Summary Statistics (5 marks)

Exploratory data analysis often involves calculating summary statistics to understand the characteristics of a dataset. Now that you have access to the global set `ENGLISH_WORDS`, you can analyze the distribution of word lengths in the English vocabulary.

Write a function `word_length_summary(word_set)` that takes a set of word strings as its argument and returns a dictionary containing summary statistics about the lengths of those words.

Specifically, the returned dictionary must contain exactly the following three key-value pairs:

- 'mean_length': A float representing the average length of all words in word_set, rounded to 2 decimal places. You can use the built-in `round()` function.
- 'max_length': An int representing the length of the longest word(s) in word_set.
- 'length_distribution': A dict mapping each word length k (an int) present in word_set to the total number of words (an int) of length k.

If word_set is empty, your function should return `None`.

In [59]:
```
SAMPLE_SET = {"cat", "dog", "elephant", "bear"}
```

Expected Output with `SAMPLE_SET` word set.

```
{
    'mean_length': 4.5,
    'max_length': 8,
    'length_distribution': {3: 2, 4: 1, 8: 1}
}
```

In [60]:
```
def word_length_summary(word_set: set) -> dict:
    stats = {
        'mean_length': None,
        'max_length': None,
        'length_distribution': None
    }
    # complete your code here
    if len(word_set) == 0:
        return None

    total_length = 0
    max_length = 0
    length_distribution = {}

    for word in word_set:
        length = len(word)
        total_length += length

        if length > max_length:
            max_length = length

        if length in length_distribution:
            length_distribution[length] += 1
        else:
            length_distribution[length] = 1

    stats['mean_length'] = round(total_length / len(word_set), 2)
    stats['max_length'] = max_length
    stats['length_distribution'] = length_distribution
    return stats
```

In [61]:
```
# Run this cell to test your word_length_summary function
do_tests(word_length_summary)
```

```
*Autograder (v3.0)*
Testing function: word_length_summary
Evaluating: word_length_summary(WORD_TEST1) ...
Returned: {'mean_length': 4.5, 'max_length': 8, 'length_distribution': {3: 2, 8:
1, 4: 1}}
Expected answer: {'mean_length': 4.5, 'max_length': 8, 'length_distribution': {3:
2, 8: 1, 4: 1}}
CORRECT :)    1 mark
Evaluating: word_length_summary(WORD_TEST1) ...
Returned: {'mean_length': 4.5, 'max_length': 8, 'length_distribution': {3: 2, 8:
1, 4: 1}}
Expected answer: {'max_length': 8, 'mean_length': 4.5, 'length_distribution': {3:
2, 8: 1, 4: 1}}
CORRECT :)    1 mark
Evaluating: word_length_summary(WORD_TEST1) ...
Returned: {'mean_length': 4.5, 'max_length': 8, 'length_distribution': {3: 2, 8:
1, 4: 1}}
Expected answer: {'mean_length': 4.5, 'max_length': 8, 'length_distribution': {3:
2, 4: 1, 8: 1}}
CORRECT :)    1 mark
Evaluating: word_length_summary(WORD_TEST2) ...
Returned: {'mean_length': 6.0, 'max_length': 6, 'length_distribution': {6: 1}}
Expected answer: {'mean_length': 6.0, 'max_length': 6, 'length_distribution': {6:
1}}
CORRECT :)    1 mark
Evaluating: word_length_summary(WORD_TEST3) ...
Returned: {'mean_length': 6.0, 'max_length': 10, 'length_distribution': {6: 6, 5:
6, 9: 3, 4: 7, 8: 4, 10: 1, 7: 2, 3: 1}}
Expected answer: {'mean_length': 6.0, 'max_length': 10, 'length_distribution':
{5: 6, 4: 7, 9: 3, 6: 6, 8: 4, 7: 2, 10: 1, 3: 1}}
CORRECT :)    1 mark
--------------------------------------------------
Total mark for 'word_length_summary' is 5 out of 5
--------------------------------------------------
```

Question 3: Generative AI Comparisons and Reflective Assessment (10 Marks)

In modern software development and data science, Generative AI (GenAI) tools are increasingly used as coding assistants. Reflecting on and evaluating AI-generated code against your own is an essential skill for identifying edge cases, identifying gaps in knowledge, and personal learning needs.

For this question, select a **free version** of GenAI tool of your choice and prompt it to solve Question 1 and Question 2. The University recommends Copilot for fairness, privacy, and data protection.

Write your response to the following questions.

3(a) GenAI Tool Specification (2 marks)

State the specific Generative AI tool used, including model version and date of use (e.g., ChatGPT-4o, Gemini 1.5 Pro, Claude 3.5 Sonnet).

GenAI Tool: ChatGPT (OpenAI) Model/Version: GPT-6(free version) Date used: 23 October 2026

3(b) Solution Comparative and Analysis (4 marks)

Copy and paste the code generated by your chosen GenAI tool for Question 1 and 2.

In [62]:
```
# Copy and paste the AI solutions here for Question 1

def available_movie_tags_AI(max_rent:int, movie_data:dict) -> list:
    # Get unique tags from movies within the rental budget and sort them.
    return sorted({
        tag
        for cost, tags in movie_data.values()
        if cost <= max_rent
        for tag in tags
    })


def recommend_movies_AI(max_cost:int, tags:list, movie_data:dict) -> list:
    # Convert the requested tags into a set.
    wanted = set(tags)

    # Select movies within budget that contain all requested tags.
    matches = [
        [name, cost, movie_tags]
        for name, (cost, movie_tags) in movie_data.items()
        if cost <= max_cost and wanted <= set(movie_tags)
    ]

    # Sort matching movies by cost, from highest to lowest.
    return sorted(matches, key=lambda m: m[1], reverse=True)
```

In [63]:
```
# Copy and paste the AI solutions here for Question 2

def is_english_word_AI(s: str) -> bool:
    # Check whether the word exists and uses an accepted capitalisation.
    return (
        s.lower() in ENGLISH_WORDS
        and (s.islower() or s.isupper() or s == s.capitalize())
    )


def password_strength_AI(password: str) -> str:
    # Reject English words and passwords containing whitespace.
    if is_english_word_AI(password) or any(c in " \t\n" for c in password):
        return "ILLEGAL"

    # Passwords shorter than 8 characters are weak.
    if len(password) < 8:
        return "WEAK"

    # Check whether the password is an English word followed by digits.
    word = password.rstrip("0123456789")
    digits = password[len(word):]

    if digits and is_english_word_AI(word):
```

```
        return "WEAK"

    # Check which character types are present.
    has_lower = any(c.islower() for c in password)
    has_upper = any(c.isupper() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(
        not c.isalnum() and c.isascii() for c in password
    )

    # Strong passwords need at least 12 characters and all four character types.
    if (len(password) >= 12 and has_lower and has_upper
            and has_digit and has_special):
        return "STRONG"

    return "MEDIUM"


def word_length_summary_AI(word_set: set) -> dict:
    # Return None if the set is empty.
    if not word_set:
        return None

    # Calculate the length of each word and count each length.
    lengths = [len(word) for word in word_set]
    distribution = {
        length: lengths.count(length)
        for length in sorted(set(lengths))
    }

    # Return the mean length, maximum length, and length distribution.
    return {
        "mean_length": round(sum(lengths) / len(lengths), 2),
        "max_length": max(lengths),
        "length_distribution": distribution
    }
```

Compare your completed solutions from Question 1 and 2 with the code generated by your chosen GenAI tool. Discuss differences in:

- Code readability and structure
- Edge-case handling (e.g., empty sets)
- Algorithmic approach and code efficienty (e.g., time/space complexity, use of built-in functions and methods).
- adherence to constraints (e.g., no external library imports)

Answer : After comparing my solutions for Questions 1 and 2 with the solutions generated by my chosen GenAI tool, I noticed differences in code structure, handling of special cases, efficiency, and the use of Python features. My solutions mainly used loops and conditional statements, whereas the AI answers used shorter expressions and built-in functions. Both approaches aim to produce the required results, but comparing them helped me understand the advantages and limitations of each approach.

1. Considering Code readability and structure, In Question 1(a), I used a for loop to check each movie's rental cost and added the tags of affordable movies to a set. I

then used sorted() to return unique tags in alphabetical order. In Question 1(b), I divided the task into checking available tags, filtering movies by budget and requested tags, and sorting the results. I used nested loops to sort movies by cost, whereas the AI solution used Python's built-in sorting function, making the code shorter and generally more efficient.

In Question 2(a), I used nested if statements to check whether a word existed in ENGLISH_WORDS and followed the correct capitalisation rules. The AI solution combined these conditions using and and or. In Question 2(b), I used a loop and Boolean variables to check for lowercase letters, uppercase letters, digits, and special characters, while the AI solution used any() for more concise checks. In Question 2(c), I calculated the word statistics in one loop, whereas the AI solution used built-in functions such as sum() and max() and created a list of word lengths. The AI-generated code also included comments for almost every line, explaining the purpose of each step. These comments improved readability, while my code focused on explaining the logic through separate steps.

2. Considering Edge-case handling, In Q1(a), my solution and the AI solution use a set to remove duplicate tags and return sorted tags; if no movies are affordable, the result is an empty list. In Q1(b), both solutions return an empty list if a requested tag is unavailable. In Q2(a), both accept lowercase, capitalised, and uppercase words but reject incorrect capitalisation. In Q2(b), both handle whitespace, short passwords, and English words followed by digits. In Q2(c), both return None for an empty set to avoid errors when calculating statistics.
3. Considering Algorithmic approach and code efficiency,my sorting method in Question 1(b) uses nested loops and can take (O(k^2)) time for (k) matching movies. The AI's sorting generally takes (O(klog k)) time, making it more efficient for larger results. In Question 2(a), both approaches use set membership to check words, which takes expected (O(1)) time. In Question 2(b), both methods examine the password characters, giving linear time complexity in the password length. In Question 2(c), my solution calculates the statistics in one pass, taking expected (O(n)) time for (n) words. The AI solution creates a list of lengths and repeatedly counts each distinct length, which can take (O(nd)) time for (d) distinct lengths. Therefore, my approach can be more efficient for calculating the distribution and avoids creating an extra list.
4. Both approaches use standard Python features without requiring additional external library imports. My solutions use loops, sets, lists, dictionaries, and conditional statements, while the AI solutions also demonstrate built-in functions such as sorted(), any(), sum(), and max(). Both must preserve the required function names, parameters, return values, and output formats to meet the coursework requirements. In conclusion, comparing my solutions with the AI-generated code helped me understand different ways to solve the same problems. My approach made the logic easier for me to follow step by step, while the AI solutions were often more concise and included comments explaining individual lines. I also learned that built-in functions can improve efficiency, but their use should be understood and evaluated. Overall, good code should be correct, readable, efficient, handle edge cases, and follow the specified constraints.

3(c) Knowledge Gap and Learning Reflection (4 marks)

Based on your comparison in Q3(b), reflect on your learning process and future technical development:

- Identify any specific built-in functions, methods, algorithmic logic, or data science concepts in the GenAI ouput that you did not initally consider or fully understand.
- Outline the practical steps you will take to master these concepts and address these knowledge gaps (e.g., consulting specifc Python documentation, practicing edge-case handling, or watch more tutorial videos)

Note: If you identified no knowledge gaps, explain how you verified that your approach was equivalent.

Answer : Comparing my solutions with the GenAI output helped me understand that the same problem can be solved in different ways. In Question 1(b), I learned how Python's built-in sort() method and lambda functions can make sorting movies by price simpler. In Question 2(a), I understood how sets can be used to check whether a word exists. In Question 2(b), I learned how the any() function can simplify checking password conditions. In Question 2(c), I realised that calculating word-length statistics in one pass can avoid creating an extra list.

To improve my programming skills, I plan to practise regularly on HackerRank, especially its Python exercises. I will use the official Python documentation to understand functions such as any(), sorted(), and lambda functions. I will also use W3Schools Python tutorials when I need simpler explanations and examples. After attempting each problem independently, I will compare alternative solutions, test different inputs and edge cases, and revisit any concepts I find difficult.

Overall, this comparison showed me that producing the correct output is only one part of programming. I also need to understand why a solution works and how to make my code clearer and easier to maintain. By using these resources and practising consistently, I hope to become more confident in solving programming problems independently and applying Python to future data science tasks.

Submission

- Coursework should be submitted via Gradescope. Link is on Minerva in the folder Submit my work.
- You should submit your edited version `cwk1.ipynb` as well as produce a PDF version of that notebook to be submitted together.
- **Do not change the name of the notebook file, `cwk1.ipynb` as the Gradescope Autograder will look for this file**